Contents

List of Figures iv

List of Tables v

1 Introduction 1
- 1.1 General . . . 1
- 1.2 Background . . . 1
- 1.3 Ideation . . . 1
- 1.4 Overview of Object Detection Models . . . 2

2 Objective and Goals 3
- 2.1 Objectives of the Project . . . 3
- 2.2 Scope of Project . . . 3

3 Requirements 4
- 3.1 Functional Requirements . . . 4
- 3.2 Software Requirements . . . 5
- 3.3 Hardware Requirements . . . 6

4 Project Plan 7
- 4.1 Datasets . . . 7
  - 4.1.1 Dataset A . . . 7
  - 4.1.2 Dataset B . . . 7
  - 4.1.3 Dataset C . . . 7
  - 4.1.4 Dataset D . . . 7
  - 4.1.5 Dataset E . . . 7
  - 4.1.6 Stanford Background Dataset [8] . . . 8
- 4.2 Evaluation Metrics . . . 9
- 4.3 Plan of Action . . . 9
  - 4.3.1 Step 1: Flag design preparation . . . 9
  - 4.3.2 Step 2: Background Dataset Collection . . . 10
  - 4.3.3 Step 3: Synthetic Dataset Generation . . . 10
  - 4.3.4 Step 4: Model Training . . . 10

4.3.5 Step 5: Performance Evaluation . . . 11
4.3.6 Step 6: Model Evaluation . . . 11
4.3.7 Step 7: Comparative Analysis . . . 11

5 Packages/Modules Used 12

6 Working Project 13
6.1 Code Implementation . . . 13
6.2 Output . . . 20
6.2.1 Annotation of synthetic images . . . 20
6.2.2 Image generated by each domain randomization technique . . . 20

7 Equations and Diagrammatic Representations 22
7.1 Equations . . . 22
7.1.1 Random Background . . . 22
7.1.2 Realistic Scale . . . 22
7.1.3 Random Position . . . 22
7.1.4 Perspective Warp . . . 23
7.1.5 Brightness Adjustment . . . 23
7.1.6 Contrast Adjustment . . . 23
7.1.7 Saturation Adjustment . . . 23
7.1.8 Gaussian Noise . . . 23
7.1.9 Shadow . . . 24
7.1.10 Blur . . . 24
7.1.11 Occlusion . . . 24
7.2 Flowchart . . . 25

8 Observations and Results 26
8.1 Model Training Results . . . 26
8.2 Model Testing Results . . . 27
8.3 Observations . . . 27

9 Testing 29

10 Conclusion 30

References 31

List of Figures

4.1 Synthetic flag image. 10

6.1 Initial annotation for synthetic DATASET A 20
6.2 Sample images from the generated synthetic datasets. 21

7.1 Summary of project workflow. This is repeated from DATASET A to DATASET D. 25

List of Tables

3.1 Summary of software requirements 5
3.2 Summary of hardware requirements 6

4.1 Dataset split for training, validation, and evaluation. 8
4.2 Evaluation metrics used in this project. 9

8.1 Training Result of YOLOv8n on Synthetic Datasets 26
8.2 Training Result of YOLOv8s on Synthetic Datasets 26
8.3 Test Results of YOLOv8n on unseen DATASET E 27
8.4 Test Results of YOLOv8s on unseen DATASET E 27
8.5 Summary of Experimental Observations 28

9.1 Implementation Challenges and Solutions 29

CHAPTER 1

INTRODUCTION

1.1 General

Computer Vision has become an emerging field in Deep Learning. Under computer vision, we have many tasks, including Classification, where a model looks at an image and predicts a class (example dog or cat), Localization;where a model locates an object in an image and draws a bounding box assigning it coordinates (x,y,width,height), Object Detection which involves classification and localization, and Image Segmentation that divides an image into distinct pixel groups, extracting object of interest from an image having multiple objects.

1.2 Background

Recent developments in computer vision have shown that synthetic data can effectively supplement real world datasets for many computer vision tasks. Domain randomization is a common technique in synthetic dataset generation. It introduces variations such as illumination, shadows, blur, occlusions, and backgrounds. These variations help deep learning models generalise more effectively on unseen data. The need for a synthetic dataset comes from the limited availability of certain datasets due to privacy or resource constraints. For example, medical records are not easily available to protect privacy of patients, or in the defence sector we need to test a newly designed technology which maybe confidential or inaccessible for large scale image collection. However, an important question arises : **Does the trained deep learning model learn the target's characteristics, or does it simply respond to certain features?** Evaluating the model on an unseen collection of images provides valuable insight into the reliability of such models.

1.3 Ideation

The motivation behind this project is the limited availability of real world images due to certain constraints. In many applications, only a digital design is available during early stages of development, making conventional dataset collection impossible. This project

investigates whether an object detector can be trained on only synthetically generated images derived from a single design. Furthermore, it aims to examine whether progressively increasing the realism of synthetic data through domain randomization improves the detector's ability to distinguish the target object from visually similar objects. By eliminating the dependence on real world image collection and manual annotation, the proposed framework offers a scalable and cost-effective solution for developing object detection models for previously unseen objects.

1.4 Overview of Object Detection Models

- **SSD** Single Shot Detector (SSD) is a deep learning-based object detection algorithm that identifies and localizes objects in an image using a single forward pass through the network. SSD detects objects using multiple feature maps.

- **Overview of YOLO** (You Only Look Once) is an object detection algorithm that came on the scene in 2015, known for its lightning-fast processing of entire images at once. YOLO predicts objects using a single grid-based view of the image.

- **YOLOv8** YOLOv8 is the newest version, taking previous iterations and making them even speedier and more accurate. The YOLO evolution includes versions like YOLOv1, v2, v3, v4, and v5, each bringing improvements like real-time processing, batch normalization, and better detection accuracy. YOLOv8 brings in cutting-edge techniques to take object detection performance even further.YOLOv8n is light and uses less memory than YOLOv8s.

- **Faster R-CNN** Faster R-CNN is a deep learning-based object detection model that identifies and localises multiple objects in an image using a unified architecture. It improves on R-CNN and Fast R-CNN by introducing a Region Proposal Network (RPN) for efficient proposal generation.

- **Transfomer based object detection** Facebook on 27 May 2020 released DETR standing for Detection Transformer as it uses transformers to detect objects.This is the first time that transformer is used for such a task of Object detection along with a Convolutional Neural network. There are other Object detection models such as the R-CNN family, YOLO(You Look Only Once) and SSD(Single Shot Detection) but none of them have ever used a transformer to achieve this task. The best part of this model is that due to the fact that it is using a transformer, it makes the architecture very simple unlike all the other techniques mentioned with has all kinds of hyperparameters and layers.DETR demonstrates significantly better performance on large objects and not on a small object which can be further improved.

CHAPTER 2

OBJECTIVE AND GOALS

2.1 Objectives of the Project

The specific objectives of this project are:

1. To create four synthetic datasets from a given flag design using real background dataset.
2. To investigate the impact of progressive domain randomization techniques on object detector performance.
3. To train and compare YOLO v8s, YOLO v8n and Faster R CNN on these datasets and evaluate using precision, recall, mAP, and time metrics.
4. To test the trained models on an unseen dataset, which is also synthetically generated, containing similar flag shapes, and a high level of domain randomization to test the models' power to generalise the flag.

2.2 Scope of Project

1. Large and extensive datasets are not easily available, especially in medical, defence and other fields because of privacy reasons and also if a new technology is introduced we don't have enough real world data to gauge it's performance. This project tries to bridge this gap by developing realistic synthetic datasets.
2. Synthetic datasets have shown to be equally effective to test model's performance on a real unseen data, but in this project we don't have any real image unlike previous studies. Hence, we investigate whether models trained on synthetic data can generalise even in total absence of real world images.

CHAPTER 3

REQUIREMENTS

3.1 Functional Requirements

The functions this system should perform are:

1. Accept a single flag design with a transparent background as input.
2. Generate 1000 synthetic images by placing object on diverse background images.
3. Progressively apply domain randomization techniques :
    - **DATASET A:** random background, realistic scale to match the background, random position.
    - **DATASET B:** random background, realistic scale, random position, perspective warp (it simulates how an object appears when viewed from different camera angles).
    - **DATASET C:** random background, realistic scale, random position, perspective warp, brightness, contrast, saturation (for different shades of orange), gaussian noise (to simulate imperfections introduced by camera sensors).
    - **DATASET D:** random background, realistic scale to match the background, random position, perspective warp, brightness, contrast, saturation, gaussian noise, shadow, blur, and occlusions (partial obstruction of an object by another object).
    - **DATASET E:** similar unseen orange flag shapes (square and triangle), random background, realistic scale to match the background, random position, perspective warp, brightness, contrast, saturation, gaussian noise, shadow, blur, and occlusions.
4. Automatically generate bounding box annotations.
5. Split all four datasets into training,validation and testing sets.

6. Train YOLOv8n, YOLOv8s and Faster R-CNN on the four datasets and after training on each dataset, test the models on an unseen **DATASET E**.

7. Evaluate trained models using mAP50, mAP50-95, Precision, Recall, training time, and inference time.

3.2 Software Requirements

Table 3.1: Summary of software requirements

S.No.	Component	Purpose
1	Python 3.10	Programming language
2	Jupyter Notebook	Development environment
3	OpenCV	Image processing and synthetic image generation
4	NumPy	Numerical computations
5	Ultralytics YOLOv8	Object detection training and inference
6	PyTorch	Deep learning framework
7	Torchvision	Faster R-CNN implementation
8	Matplotlib	Visualization of results
9	Pandas	Performance analysis
10	TQDM	Progress monitoring during dataset generation
11	CUDA	GPU acceleration

3.3 Hardware Requirements

Table 3.2: Summary of hardware requirements

S.No.	Component	Specification
1	Processor	Intel Core i5 (8th Gen)
2	RAM	8 GB
3	Storage	20 GB free disk space
4	GPU	NVIDIA GPU (4 GB VRAM)
5	CUDA	CUDA 11.8
6	Operating System	Windows 11

CHAPTER 4

PROJECT PLAN

4.1 Datasets

4.1.1 Dataset A

Contains 1000 samples combined with Stanford Background Dataset [8], introducing random background, realistic scale to match the background, random position.

4.1.2 Dataset B

Contains 1000 samples combined with Stanford Background Dataset [8], random background, realistic scale, random position, perspective warp (it simulates how an object appears when viewed from different camera angles).

4.1.3 Dataset C

Contains 1000 samples combined with Stanford Background Dataset [8], random background, realistic scale, random position, perspective warp, brightness, contrast, saturation (for different shades of orange), gaussian noise (to simulate imperfections introduced by camera sensors).

4.1.4 Dataset D

Contains 1000 samples combined with Stanford Background Dataset [8], random background, realistic scale to match the background, random position, perspective warp, brightness, contrast, saturation, gaussian noise, shadow, blur, and occlusions (partial obstruction of an object by another object).

4.1.5 Dataset E

Contains 1000 samples combined with Stanford Background Dataset [8], similar unseen orange flag shapes (square and triangle), random background, realistic scale to match the

background, random position, perspective warp, brightness, contrast, saturation, gaussian noise, shadow, blur, and occlusions.

4.1.6 Stanford Background Dataset [8]

This dataset was originally created to evaluate geometric and semantic scene understanding of State of The Art Scene understanding models using less data. The dataset contains 715 images chosen from public datasets: LabelMe, MSRC, PASCAL VOC and Geometric Context. The dataset contains many outdoor scenes (Our flag can be spotted in outdoors) with atleast one foreground object (This can help in our flag project for better object localisation and instance identification.).

Table 4.1: Dataset split for training, validation, and evaluation.

Dataset	Total	Train	Val	Test	Role
Dataset A	1,000	700	200	100	Train + Test
Dataset B	1000	700	200	100	Train + Test
Dataset C	1000	700	200	100	Train + Test
Dataset D	1000	700	200	100	Train + Test
Dataset E	500	0	0	500	Test only
Stanford Background Dataset [8]	715	0	0	0	Train only

4.2 Evaluation Metrics

Table 4.2: Evaluation metrics used in this project.

Metric	Formula	Interpretation
Precision	$\text{Precision} = \frac{TP}{TP+FP}$	Precision measures the proportion of predicted positive detections that are actually correct.
Recall	$\text{Recall} = \frac{TP}{TP+FN}$	Recall measures the proportion of actual objects that the detector successfully identifies.
mAP@0.5	$mAP@0.5 = \frac{1}{N}\sum_{i=1}^{N} AP_i$	Mean Average Precision computed using an IoU threshold of 0.5.
mAP@0.5:0.95	$mAP@0.5:0.95 = \frac{1}{N}\sum_{k=0}^{N-1} AP_{0.5+0.05k}$	Mean Average Precision averaged over IoU thresholds from 0.50 to 0.95, providing a stricter evaluation of localization accuracy.
Training Time	$T_{\text{train}} = t_{\text{end}} - t_{\text{start}}$	Training time measures the computational cost of learning.
Inference Time	$T_{\text{inference}} = \frac{T_{\text{total}}}{N}$	Lower inference time indicates faster object detection and is important for real-time applications.

4.3 Plan of Action

4.3.1 Step 1: Flag design preparation

Given flag design has been created in word document . Dimensions are in inches. The width and length of the flag is in ratio of 4:3.Remove the background of the flag image.

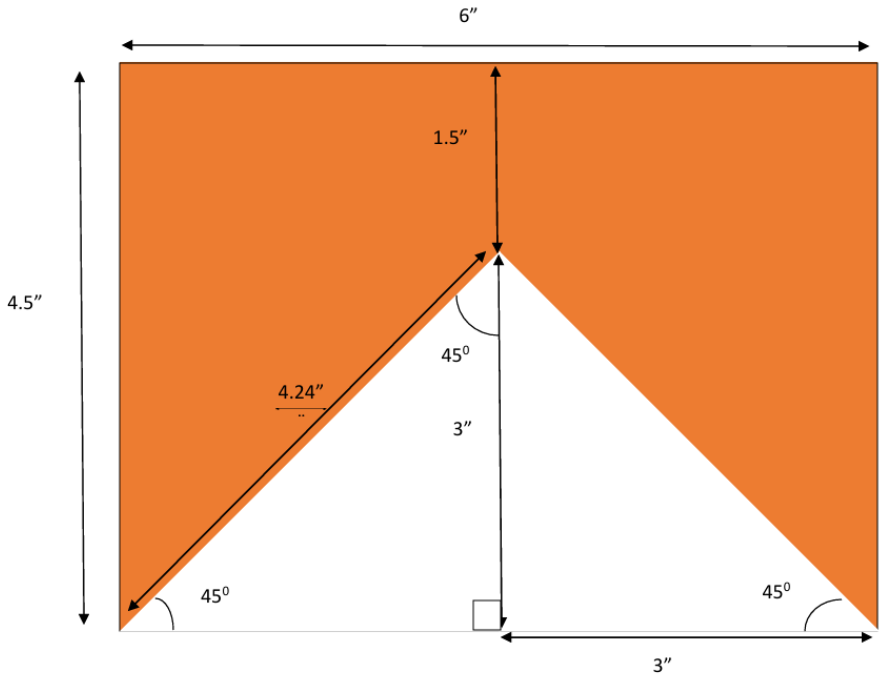

Figure 4.1: Synthetic flag image.

4.3.2 Step 2: Background Dataset Collection

Collect diverse natural scene images to serve as background environments.

4.3.3 Step 3: Synthetic Dataset Generation

- Dataset A: Basic object placement on random backgrounds collected in Step 2.
- Dataset B: Apply geometric transformations (e.g., perspective warp, scaling, rotation).
- Dataset C: Introduce photometric variations (e.g., brightness, contrast, Gaussian noise).
- Dataset D: Apply advanced augmentations including shadows, blur, and partial occlusions.
- Dataset E: Synthetic test dataset containing unseen orange flags of similar shape like triangle or square.

4.3.4 Step 4: Model Training

Train three object detection models for all four datasets: YOLOv8n, YOLOv8s, Faster R-CNN

4.3.5 Step 5: Performance Evaluation

Evaluate the trained models using : Precision, Recall, mAP@0.5, mAP@0.5:0.95, Training time, Inference time

4.3.6 Step 6: Model Evaluation

After training models on each dataset, test the trained models on dataset E and analyze evaluation metrics.

4.3.7 Step 7: Comparative Analysis

Compare the performance of all trained models based on: precision, recall, mAP, computational efficiency, model complexity

CHAPTER 5

PACKAGES/MODULES USED

Given below is a brief description of all packages and modules used in the project :

- **OpenCV (cv2)** was used for synthetic image generation, including image loading, resizing, perspective transformations, geometric augmentations, and automatic annotation generation.
- **NumPy** provided efficient array operations and mathematical functions required for image manipulation and random augmentation.
- **PyTorch** served as the primary deep learning framework for implementing and training the Faster R-CNN model.
- **torchvision** supplied pretrained Faster R-CNN architectures and image preprocessing utilities.
- **matplotlib** was used to visualize training loss, precision, recall, mAP, and detection results.
- **ultralytics** library was used to train, validate, and evaluate the YOLOv8n and YOLOv8s object detection models.
- **Pandas** was employed for tabulating and comparing evaluation metrics across different models.
- **tqdm** library provided real-time progress bars during synthetic dataset generation and training, improving monitoring of long-running tasks.
- **os** module handled file paths, directory creation, and automated dataset organization.
- **random** module generated diverse object placements, scaling factors, and background selections during dataset synthesis.
- **time** module was used to compute total training time and average inference time for performance comparison.

CHAPTER 6

WORKING PROJECT

This section provides the code snippets and output obtained while carrying out the project.

6.1 Code Implementation

Given below is a complete code to generate DATASET A. It involves loading the flag design whose background has been removed so that it can be blended with our real background images (Stanford Background Dataset [8]). It also splits dataset into test, train and val sets. It also introduces initial domain randomization as discussed earlier. Rest of the datasets B,C,D,E have been created in a similar fashion.

Listing 6.1: Synthetic DATASET A generation

```
import cv2
import os
import random
import numpy as np
from tqdm import tqdm


# PATHS


FLAG_PATH = "/content/drive/MyDrive/Flag design background
    removed.png"
BACKGROUND_DIR = "/content/drive/MyDrive/images"

OUTPUT_DIR = "dataset_A"

NUM_IMAGES = 1000


```

```
# CREATE TRAIN/VAL/TEST

for split in ["train", "val", "test"]:

    os.makedirs(
        os.path.join(
            OUTPUT_DIR,
            "images",
            split
        ),
        exist_ok=True
    )

    os.makedirs(
        os.path.join(
            OUTPUT_DIR,
            "labels",
            split
        ),
        exist_ok=True
    )


# LOAD FLAG


flag = cv2.imread(
    FLAG_PATH,
    cv2.IMREAD_UNCHANGED
)

if flag is None:
    raise ValueError("Could not load flag image")

if flag.shape[2] != 4:
    raise ValueError(
        "Flag PNG must have an alpha channel"
    )


# LOAD BACKGROUNDS
```

```
60
61
62  backgrounds = [
63      os.path.join(BACKGROUND_DIR, f)
64      for f in os.listdir(BACKGROUND_DIR)
65  ]
66
67  if len(backgrounds) == 0:
68      raise ValueError(
69          "No background images found"
70      )
71
72
73  # CREATE SPLIT MAP
74
75
76  indices = list(range(NUM_IMAGES))
77
78  random.shuffle(indices)
79
80  train_end = int(0.7 * NUM_IMAGES)
81  val_end = int(0.9 * NUM_IMAGES)
82
83  split_map = {}
84
85  for idx in indices[:train_end]:
86      split_map[idx] = "train"
87
88  for idx in indices[train_end:val_end]:
89      split_map[idx] = "val"
90
91  for idx in indices[val_end:]:
92      split_map[idx] = "test"
93
94
95  # GENERATION LOOP
96
97
98  for i in tqdm(range(NUM_IMAGES)):
99
100     split = split_map[i]
```

```
101
102
103     # RANDOM BACKGROUND
104
105
106     bg_path = random.choice(backgrounds)
107
108     bg = cv2.imread(bg_path)
109
110     if bg is None:
111         continue
112
113     bg_h, bg_w = bg.shape[:2]
114
115
116     # REALISTIC SCALE
117
118
119     scale = np.random.beta(2, 5)
120     scale = 0.05 + scale * 0.45
121
122     aspect_ratio = flag.shape[0] / flag.shape[1]
123
124     new_w = int(bg_w * scale)
125     new_h = int(new_w * aspect_ratio)
126
127     new_w = max(new_w, 32)
128     new_h = max(new_h, 32)
129
130     if new_w >= bg_w:
131         continue
132
133     if new_h >= bg_h:
134         continue
135
136     resized_flag = cv2.resize(
137         flag,
138         (new_w, new_h),
139         interpolation=cv2.INTER_AREA
140     )
141
```

```
142
143     # RANDOM POSITION
144
145
146     x = random.randint(
147         0,
148         bg_w - new_w
149     )
150
151     y = random.randint(
152         0,
153         bg_h - new_h
154     )
155
156
157
158     rgb = resized_flag[:, :, :3]
159
160     alpha = resized_flag[:, :, 3] / 255.0
161
162     for c in range(3):
163
164         bg[
165             y:y+new_h,
166             x:x+new_w,
167             c
168         ] = (
169             alpha * rgb[:, :, c]
170             +
171             (1 - alpha)
172             * bg[
173                 y:y+new_h,
174                 x:x+new_w,
175                 c
176             ]
177         )
178
179
180     # TIGHT BOUNDING BOX
181
182
```

```
    alpha_mask = resized_flag[:, :, 3]

    ys, xs = np.where(alpha_mask > 10)

    if len(xs) == 0:
        continue

    xmin = xs.min()
    xmax = xs.max()

    ymin = ys.min()
    ymax = ys.max()

    box_xmin = x + xmin
    box_xmax = x + xmax

    box_ymin = y + ymin
    box_ymax = y + ymax


    # ANNOTATION


    x_center = (
        (box_xmin + box_xmax) / 2
    ) / bg_w

    y_center = (
        (box_ymin + box_ymax) / 2
    ) / bg_h

    width = (
        box_xmax - box_xmin
    ) / bg_w

    height = (
        box_ymax - box_ymin
    ) / bg_h


    # SAVE IMAGE
```

```
224
225
226     img_name = f"img_{i}.jpg"
227
228     cv2.imwrite(
229         os.path.join(
230             OUTPUT_DIR,
231             "images",
232             split,
233             img_name
234         ),
235         bg
236     )
237
238
239     # SAVE LABEL
240
241
242     label_name = f"img_{i}.txt"
243
244     with open(
245         os.path.join(
246             OUTPUT_DIR,
247             "labels",
248             split,
249             label_name
250         ),
251         "w"
252     ) as f:
253
254         f.write(
255             f"0 {x_center:.6f} {y_center:.6f} {width:.6f} {
                    height:.6f}"
256         )
257
258 print("Done!")
```

6.2 Output

6.2.1 Annotation of synthetic images

As we can see in Figure 6.1, the code for initial synthetic data generation had loose bounding box. This can hinder model's learning and hence, the code has been modified to generate tight bounding boxes (Listing 6.1).

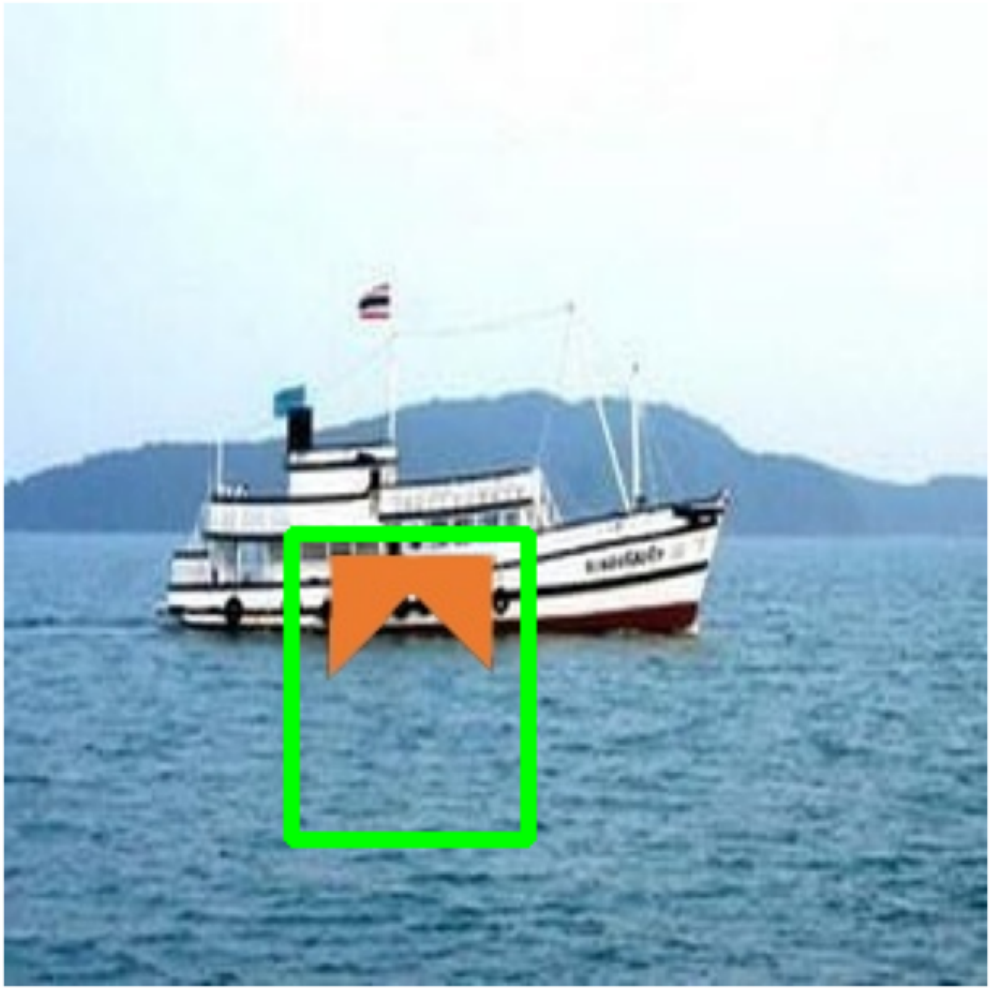

Figure 6.1: Initial annotation for synthetic DATASET A

6.2.2 Image generated by each domain randomization technique

In chapter 4 we discussed domain randomization techniques applied to each dataset. Here, we display one sample from each dataset to demonstrate how our synthetic images look after every domain randomization technique. The model learns from these images and the model is then tested on an unseen dataset (DATASET E) containing similar shape flags.

(a) Dataset A

(b) Dataset B

(c) Dataset C

(d) Dataset D

(e) Unseen Dataset E (Occlusions)

Figure 6.2: Sample images from the generated synthetic datasets.

CHAPTER 7

EQUATIONS AND DIAGRAMMATIC REPRESENTATIONS

7.1 Equations

Given below are explanation and equations defining various domain randomization techniques used in this project

7.1.1 Random Background

Placing the target object onto a randomly chosen natural scene image. This increases scene diversity and reduces overfitting to a particular background.

$$B \sim \mathscr{U}(B_1, B_2, \ldots, B_n)$$

where, B = selected background image, U = uniform random selection, n = total number of available backgrounds

7.1.2 Realistic Scale

The object is resized using a random scaling factor while preserving its aspect ratio to simulate different distances from the camera.

$$W' = sW, \qquad H' = sH$$

Where, W, H = original width and height, W', H'= resized dimensions, s = scaling factor

7.1.3 Random Position

The object is placed at a randomly selected location within the background image while ensuring that it remains entirely inside the image boundaries.

$$x \sim U(0, W_b - W_o), \qquad y \sim U(0, H_b - H_o)$$

Where W_b and H_b represent the width and height of the background image, respectively, W_o and H_o represent the width and height of the object image, and $U(a,b)$ denotes a uniform distribution over the interval $[a,b]$. The variables x and y correspond to the randomly selected horizontal and vertical placement coordinates of the object.

7.1.4 Perspective Warp

Perspective warp simulates different viewing angles by applying a projective transformation to the object.

$$\mathbf{x}' = H\mathbf{x}$$

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

7.1.5 Brightness Adjustment

Brightness augmentation modifies pixel intensities by adding a constant value.

$$I' = I + \beta$$

Where, I = original image, I' = adjusted image, β = brightness offset

7.1.6 Contrast Adjustment

Contrast augmentation changes the intensity difference between pixels.

$$I' = \alpha I$$

Where, α = contrast scaling factor

7.1.7 Saturation Adjustment

Saturation controls the intensity of colors in the image, making them more vivid or muted.

$$S' = \gamma S$$

Where, S = original saturation, S' = adjusted saturation, γ = saturation scaling factor

7.1.8 Gaussian Noise

Gaussian noise randomly perturbs pixel intensities according to a normal distribution to simulate sensor noise.

$$I' = I + N(0, \sigma^2)$$

Where, N(0,σ^2) = Gaussian noise, σ^2 = variance

7.1.9 Shadow

Shadow augmentation simulates partial illumination changes by reducing pixel intensities in selected regions.

$$I' = \lambda I$$

Where, $0 < \lambda < 1$ is the shadow intensity factor.

7.1.10 Blur

Blur simulates out-of-focus or motion effects by smoothing pixel intensities using a Gaussian filter.

$$I' = I * G$$

Where, $*$ = convolution, G = Gaussian kernel Gaussian kernel:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

7.1.11 Occlusion

Occlusion partially hides the target object by covering a randomly selected region, simulating real-world obstruction.

$$I'(x,y) = \begin{cases} O(x,y), & (x,y) \in R \\ I(x,y), & \text{otherwise} \end{cases}$$

Where, R = occluded region, O(x,y) = occluding object or mask, I(x,y) = original image

7.2 Flowchart

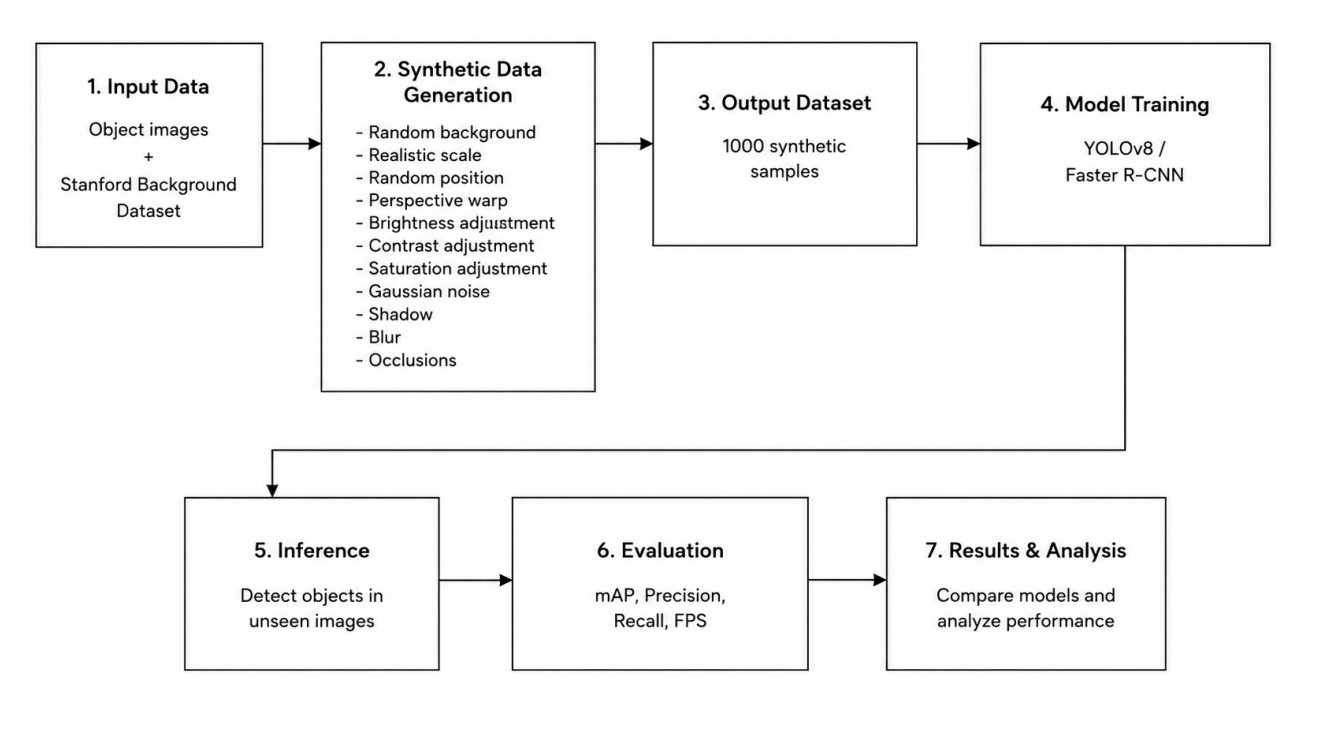

Figure 7.1: Summary of project workflow. This is repeated from DATASET A to DATASET D.

CHAPTER 8

OBSERVATIONS AND RESULTS

8.1 Model Training Results

Table 8.1: Training Result of YOLOv8n on Synthetic Datasets

Dataset	Precision	Recall	mAP @0.50	mAP @0.50:0.95	Training Time (min)	Inference Time (ms)
Dataset A	0.9995	1.0000	0.9950	0.9770	12.34	12.19
Dataset B	0.9994	1.0000	0.9950	0.9125	11.52	9.93
Dataset C	0.9994	1.0000	0.9950	0.9312	12.18	10.46
Dataset D	1.0000	0.9893	0.9936	0.8526	11.41	10.01

Table 8.2: Training Result of YOLOv8s on Synthetic Datasets

Dataset	Precision	Recall	mAP @0.50	mAP @0.50:0.95	Training Time (min)	Inference Time (ms)
Dataset A	0.9995	1.0000	0.9950	0.9790	13.81	13.78
Dataset B	0.9997	1.0000	0.9950	0.9372	19.32	15.59
Dataset C	0.9994	1.0000	0.9950	0.9328	13.49	13.93
Dataset D	0.9955	0.9800	0.9847	0.8330	13.83	14.29

8.2 Model Testing Results

Table 8.3: Test Results of YOLOv8n on unseen DATASET E

Dataset	Precision (%)	Recall (%)	mAP @0.50	mAP @0.50:0.95
Dataset A	46.49	46.40	0.3875	0.2104
Dataset B	61.72	59.00	0.5286	0.3516
Dataset C	90.49	49.46	0.5813	0.4018
Dataset D	95.76	81.35	0.9104	0.5589

Table 8.4: Test Results of YOLOv8s on unseen DATASET E

Dataset	Precision (%)	Recall (%)	mAP @0.50	mAP @0.50:0.95
Dataset A	47.18	48.60	0.3848	0.2131
Dataset B	66.80	64.80	0.6060	0.4073
Dataset C	64.10	58.20	0.5305	0.3614
Dataset D	93.45	76.20	0.8801	0.5622

8.3 Observations

Table 8.5 presents the observations in metrics change after each dataset A to D is introduced to domain randomization under identical training conditions.

Table 8.5: Summary of Experimental Observations

Observation	Reasoning
All models achieved near-perfect training performance on Datasets A–D.	The synthetic datasets were well-annotated and sufficiently representative for learning the target object.
Training performance remained similar across Datasets A–D.	The task involved a single object class, allowing all models to fit the training data effectively despite increasing augmentation complexity.
Testing performance on unseen Dataset E improved from Dataset A to Dataset D.	Progressive geometric and photometric augmentations improved the models' ability to generalize to unseen data.
Models trained on Dataset D achieved the highest overall detection performance on Dataset E.	The inclusion of perspective transformations, illumination variations, blur, shadows, Gaussian noise, and occlusions enhanced robustness to real-world variations.
YOLOv8s produced slightly higher detection accuracy than YOLOv8n.	Its larger network capacity enabled better feature extraction and localization.
YOLOv8n required less training and inference time than YOLOv8s.	The lightweight architecture contains fewer parameters, resulting in lower computational complexity.
mAP@0.50:0.95 was consistently lower than mAP@0.50.	Higher IoU thresholds require more accurate bounding-box localization, making this metric more challenging to optimize.

CHAPTER 9

TESTING

Table 9.1: Implementation Challenges and Solutions

Sr. No.	Challenge	Reason / Solution
1	Unrealistic appearance of the flag	The object initially appeared artificially pasted onto the background. Perspective transformation, alpha blending, brightness and contrast adjustment, Gaussian noise, blur, shadows, and occlusions were incorporated to improve visual realism.
2	Incorrect bounding box annotations after augmentations	Image transformations altered the object boundaries. Bounding boxes were generated automatically from the transformed alpha mask to ensure accurate annotations.
3	Poor generalization to unseen images	Models trained on simple synthetic images performed poorly on unseen data. Progressive domain randomization (Datasets A–D) introduced increasing geometric and photometric variations, improving generalization.
4	False detections on visually similar objects	The detector occasionally misclassified objects with similar shapes. An additional evaluation dataset containing similar-looking triangular and square flags was created to assess detector specificity.
5	Long training time for multiple experiments	Training three object detection models on four datasets required considerable computation. Faster R-CNN required approximately one hour of training because of its two-stage detection architecture. Lightweight YOLOv8 models reduced both training and inference time while maintaining high detection performance.
6	Choice of object detection models	Faster R-CNN required significantly longer training time; therefore, YOLOv8 models were selected for detailed evaluation. Transformer-based detectors were not considered because they introduced unnecessary complexity for the scope of this study.

CHAPTER 10

CONCLUSION

It is with immense gratitude that this project was successfully completed, beginning with only a single digital flag template and no access to real-world training images. Throughout the course of this work, various concepts related to computer vision, synthetic data generation, image augmentation, and deep learning-based object detection were studied and implemented. The project provided valuable practical experience in applying theoretical knowledge to solve a real world problem.

During the implementation, several challenges were encountered, including generating visually realistic synthetic images, producing accurate annotations after geometric transformations, and improving the model's ability to generalize to unseen environments. These challenges were addressed through progressive domain randomization using geometric and photometric augmentations such as perspective transformations, brightness and contrast variations, Gaussian noise, blur, shadows, and occlusions. The experience highlighted the importance of careful dataset design in developing robust object detection models.

The proposed framework successfully generated multiple synthetic datasets of increasing complexity and evaluated their impact on the performance of YOLOv8n, YOLOv8s, and Faster R-CNN. Experimental results demonstrated that models trained on progressively augmented datasets achieved improved robustness when evaluated on an unseen synthetic dataset. In particular, Dataset D provided the best balance between synthetic diversity and detection performance, while YOLOv8s achieved the highest balance of overall precision, recall, and mAP, YOLOv8n offered superior computational efficiency.

This project demonstrates that a carefully designed synthetic data generation pipeline can serve as an effective alternative when real world annotated datasets are unavailable. The models can be trained to generalise using these synthetic datasets. The proposed methodology may be extended to other custom object detection applications where acquiring real training data is difficult, expensive, or impractical. Future work may focus on incorporating diffusion-based image generation.

BIBLIOGRAPHY

[1] F. Gaspar, D. Carreira, N. Rodrigues, R. Miragaia, J. Ribeiro, P. Costa, and A. Pereira, "Synthetic image generation for effective deep learning model training for ceramic industry applications," *Engineering Applications of Artificial Intelligence*, vol. 143, pp. 110019, 2025.

[2] D. Chen, X. Qi, Y. Zheng, Y. Lu, Y. Huang, and Z. Li, "Synthetic data augmentation by diffusion probabilistic models to enhance weed recognition," *Computers and Electronics in Agriculture*, vol. 216, Art. 108517, 2024.

[3] Y. Liu *et al.*, "High-resolution X-ray welding defect dataset generation based on data synthesis and augmentation for object detection," *Engineering Applications of Artificial Intelligence*, vol. 133, Art. 108379, 2024.

[4] I. Gräßler, M. Hieb, "Creating Synthetic Datasets for Deep Learning used in Machine Vision," *Procedia CIRP*, vol. 126, pp. 981–986, 2024.

[5] A. Tripathi, K. Tyagi, A. Agrawal, A. Tyagi, and C. Jawahar, "Learning to Generate Synthetic Data via Compositing," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10353–10362, 2019.

[6] Amit Yadav, "What is Gaussian Noise and Why It's Useful?" Medium, 2025. [Online]. Available: https://medium.com/@amit25173/what-is-gaussian-noise-and-why-its-useful-b3c50dd14628

[7] Sowmiya Narayanan Govindaraj, "What is Perspective Warping?" Medium, 2020. [Online]. Available: https://pub.towardsai.net/what-is-perspective-warping-opencv-and-python-750e7a13d386

[8] S. Gould, R. Fulton, and D. Koller, "Decomposing a Scene into Geometric and Semantically Consistent Regions," in *Proc. IEEE 12th International Conference on Computer Vision (ICCV)*, pp. 1–8, 2009.